

In [8]:

```
1  #Digital Differential Algorithm
2  #input: (x1,y1) to (x2,y2):
3  x1 = int(input("x1: "))
4  y1 = int(input("y1: "))
5  x2 = int(input("x2: "))
6  y2 = int(input("y2: "))
7
8  dely = y2 - y1
9  delx = x2 - x1
10
11 m = dely / delx
12 print("m=",m)
13
14 upx = x1
15 upy = y1
16 if m>= 0 and m<= 1:
17     upx = 1
18     upy = m
19 elif m>= -1 and m<=0:
20     upx = -1
21     upy = -m
22 elif m>1:
23     upx = 1/m
24     upy = 1
25 elif m<-1:
26     upx = -1/m
27     upy = -1
28
29 print("upx=",upx)
30 print("upy=",upy)
31
32 x = x1
33 y = y1
34
35 while (1):
36     print('(' + str(round(x)) + ',' + str(round(y)) + ')')
37     x = x + upx
38     y = y + upy
39
40     if round(x)==x2 and round(y)==y2:
41         print('(' + str(round(x)) + ',' + str(round(y)) + ')')
42         break
```

```
x1: 2
y1: 3
x2: 6
y2: 15
m= 3.0
upx= 0.3333333333333333
upy= 1
(2,3)
(2,4)
(3,5)
(3,6)
(3,7)
(4,8)
(4,9)
(4,10)
(5,11)
(5,12)
(5,13)
(6,14)
(6,15)
```

```

In [2]: 1 #bresenhams algorithm
2 x1 = int(input("x1: "))
3 y1 = int(input("y1: "))
4 x2 = int(input("x2: "))
5 y2 = int(input("y2: "))
6
7 dely = y2 - y1
8 delx = x2 - x1
9
10 ipo = (2*dely) - delx
11 print("initial point=", ipo)
12
13 x = x1
14 y = y1
15
16 while(1):
17     if ipo>=0:
18         ipo = ipo + (2*dely) - (2*delx)
19         x = x + 1 #round function not needed
20         y = y + 1
21         print('(' + str(round(x)) + ',' + str(round(y)) + ')')
22     elif ipo<0:
23         ipo = ipo + (2*dely)
24         x = x + 1
25         print('(' + str(round(x)) + ',' + str(round(y)) + ')')
26     if round(x)==x2 and round(y)==y2:
27         break

```

```

x1: 1
y1: 1
x2: 8
y2: 5
initial point= 1
(2,2)
(3,2)
(4,3)
(5,3)
(6,4)
(7,4)
(8,5)

```

```
In [5]: 1 #mid point circle
2 r = int(input("insert r: "))
3 x0 = 0
4 y0 = r
5
6 ipo = 1 - r
7 print("initial point: ", ipo)
8
9 x = x0
10 y = y0
11
12 while(1):
13     if ipo<0:
14         x = x + 1
15         ipo = ipo + (2*x) + 1
16         print('(' + str(x) + ',' + str(y) + ')')
17     elif ipo>=0:
18         x = x + 1
19         y = y - 1
20         ipo = ipo + (2*x) + 1 - (2*y)
21         print('(' + str(x) + ',' + str(y) + ')')
22
23     if x >= y:
24         break
25
```

```
insert r: 5
initial point: -4
(1,5)
(2,5)
(3,4)
(4,3)
```

```
In [10]: 1 #3D translation
2 n = int(input('enter no. of co_ordinates: '))
3
4 x = []
5 y = []
6 z = []
7
8 for i in range(n):
9     xi = int(input())
10    yi = int(input())
11    zi = int(input())
12
13    x.append(xi)
14    y.append(yi)
15    z.append(zi)
16
17 tx = int(input('Enter tx:'))
18 ty = int(input('Enter ty:'))
19 tz = int(input('Enter tz:'))
20
21 for i in range(n):
22     x[i] += tx
23     y[i] += ty
24     z[i] += tz
25
26 for i in range(n):
27     print('(' + str(x[i]) + ',' + str(y[i]) + ',' + str(z[i]) + ')')
28
```

enter no. of co_ordinates: 2

1

4

2

4

3

2

Enter tx:2

Enter ty:3

Enter tz:2

(3,7,4)

(6,6,4)

```

In [4]: 1 #3d rotation
2 import math
3 n = int(input('enter no of co-ordinates: '))
4
5 x = []
6 y = []
7 z = []
8 r = math.radians(int(input("Enter theta: ")))
9 a = input("type axis x/y/z: ")
10
11 for i in range(n):
12     xi = int(input('x: '))
13     yi = int(input('y: '))
14     zi = int(input('z: '))
15
16     x.append(xi)
17     y.append(yi)
18     z.append(zi)
19
20 if a == 'x':
21     for i in range(n):
22         y[i] = y[i]*math.cos(r) - z[i]*math.sin(r)
23         z[i] = y[i]*math.sin(r) + z[i]*math.cos(r)
24
25 elif a == 'y':
26     for i in range(n):
27         x[i] = x[i]*math.cos(r) + z[i]*math.sin(r)
28         z[i] = z[i]*math.cos(r) - x[i]*math.sin(r)
29
30 if a == 'z':
31     for i in range(n):
32         x[i] = x[i]*math.cos(r) - y[i]*math.sin(r)
33         y[i] = x[i]*math.sin(r) + y[i]*math.cos(r)
34
35 for i in range(n):
36     print('(' + str(x[i]) + ',' + str(y[i]) + ',' + str(z[i]) + ')')
37

```

enter no of co-ordinates: 2

Enter theta: 20

type axis x/y/z: 2

x: 2

y: 2

z: 2

x: 2

y: 2

z: 2

(2,2,2)

(2,2,2)

```

In [5]: 1  #3D shearing
        2
        3  import math
        4  n = int(input('enter no of co-ordinates: '))
        5
        6  x = []
        7  y = []
        8  z = []
        9
       10  shx = int(input("Shx: "))
       11  shy = int(input("Shy: "))
       12  shz = int(input("Shz: "))
       13
       14  a = input("type axis x/y/z: ")
       15
       16  for i in range(n):
       17      xi = int(input('x: '))
       18      yi = int(input('y: '))
       19      zi = int(input('z: '))
       20
       21      x.append(xi)
       22      y.append(yi)
       23      z.append(zi)
       24
       25  if a == 'x':
       26      for i in range(n):
       27          y[i] = y[i] + x[i]*shy
       28          z[i] = z[i] + x[i]*shz
       29
       30  elif a == 'y':
       31      for i in range(n):
       32          x[i] = x[i] + y[i]*shx
       33          z[i] = z[i] + y[i]*shz
       34
       35  if a == 'z':
       36      for i in range(n):
       37          x[i] = x[i] + z[i]*shx
       38          y[i] = y[i] + z[i]*shy
       39
       40  for i in range(n):
       41      print('(' + str(x[i]) + ',' + str(y[i]) + ',' + str(z[i]) + ')')
       42

```

enter no of co-ordinates: 2

Shx: 2

Shy: 3

Shz: 2

type axis x/y/z: x

x: 2

y: 1

z: 2

x: 1

y: 1

z: 1

(2,7,6)

(1,4,3)

```
In [6]: 1 # Scaling:
2
3 n = int(input("Enter number of coordinates: "))
4
5 x = []
6 y = []
7 z = []
8
9 for i in range(n):
10     ix = int(input("Enter x: "))
11     iy = int(input("Enter y: "))
12     iz = int(input("Enter z: "))
13
14     x.append(ix)
15     y.append(iy)
16     z.append(iz)
17
18 sx = int(input("Enter sx: "))
19 sy = int(input("Enter sy: "))
20 sz = int(input("Enter sz: "))
21
22 for i in range(n):
23     x[i] *= sx
24     y[i] *= sy
25     z[i] *= sz
26
27 for i in range(n):
28     print(f"New coordinates {i + 1}: ({x[i]}, {y[i]}, {z[i]})")
```

```
Enter number of coordinates: 1
Enter x: 2
Enter y: 1
Enter z: 2
Enter sx: 1
Enter sy: 1
Enter sz: 1
New coordinates 1: (2, 1, 2)
```

```
In [7]: 1 # Reflection: x-axis/yz plane, y-axis/xz plane, and z-axis/xy plane
2
3 n = int(input("Enter number of coordinates: "))
4
5 x = []
6 y = []
7 z = []
8
9 for i in range(n):
10     ix = int(input("Enter x: "))
11     iy = int(input("Enter y: "))
12     iz = int(input("Enter z: "))
13
14     x.append(ix)
15     y.append(iy)
16     z.append(iz)
17
18 r = input("Select(x/y/z): ")
19
20 if r == 'x':
21     for i in range(n):
22         x[i] = -x[i]
23 elif r == 'y':
24     for i in range(n):
25         y[i] = -y[i]
26 elif r == 'z':
27     for i in range(n):
28         z[i] = -z[i]
29
30 for i in range(n):
31     print(f"New coordinates {i + 1}: ({x[i]}, {y[i]}, {z[i]})")
```

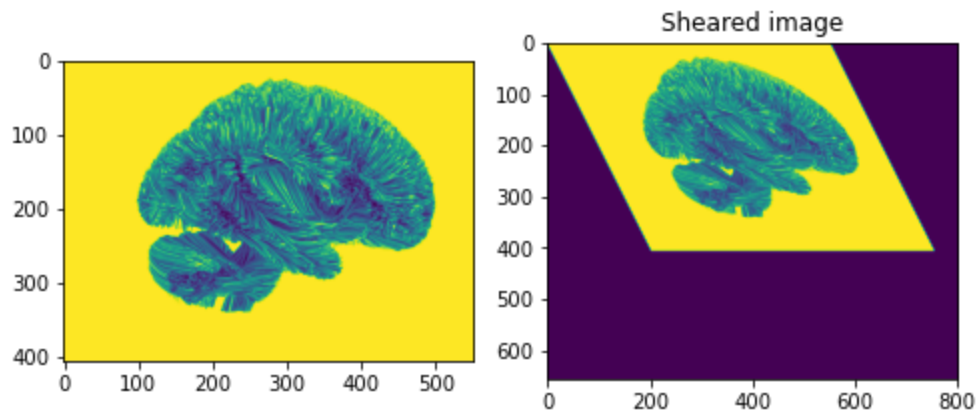
```
Enter number of coordinates: 1
Enter x: 2
Enter y: 1
Enter z: 1
Select(x/y/z): x
New coordinates 1: (-2, 1, 1)
```

```

In [8]: 1 #2D shearing x co-ordinate
2 import matplotlib.pyplot as plt
3 import matplotlib.image as img
4 import numpy as np
5 import cv2
6 import math
7
8 img = cv2.imread("../thes.png",0)
9
10 w = img.shape[1] + 250
11 h = img.shape[0] + 250
12
13 shearX = float(input("Shear value: "))
14 shearY = 0
15
16 matrix = np.array([[1, shearX, 0],
17                   [0, 1, shearY]], dtype=np.float32)
18
19 sheared_image = cv2.warpAffine(img, matrix, (w, h))
20
21 fig, axis = plt.subplots(1, 2, figsize=(7, 4))
22
23 axis[0].imshow(img)
24 axis[1].imshow(sheared_image)
25 axis[1].set_title('Sheared image')
26
27 plt.tight_layout()
28 plt.show()
29

```

Shear value: 0.5

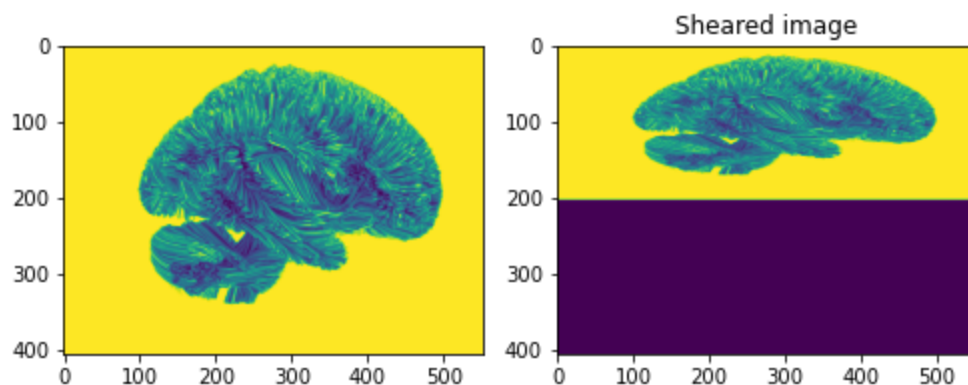


```

In [9]: 1 #2D shearing y co-ordinate
2 import matplotlib.pyplot as plt
3 import matplotlib.image as img
4 import numpy as np
5 import cv2
6 import math
7
8 img = cv2.imread("../thes.png",0)
9
10 w = img.shape[1]
11 h = img.shape[0]
12
13 shearX = 0
14 shearY = float(input("Shear value: "))
15
16 matrix = np.array([[1, 0, shearX],
17                    [0, shearY, 1]], dtype=np.float32)
18
19 sheared_image = cv2.warpAffine(img, matrix, (w, h))
20
21 fig, axis = plt.subplots(1, 2, figsize=(7, 4))
22
23 axis[0].imshow(img)
24 axis[1].imshow(sheared_image)
25 axis[1].set_title('Sheared image')
26
27 plt.tight_layout()
28 plt.show()

```

Shear value: 0.5



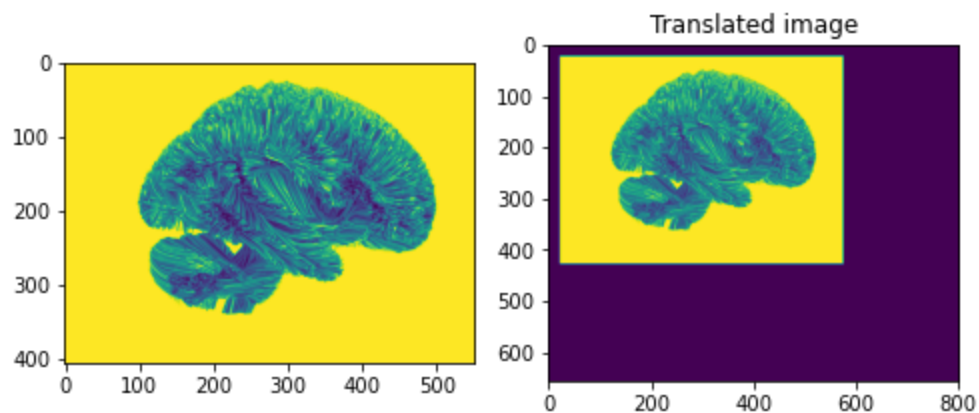
```

In [10]: 1 #2d translation
2
3 #translation
4 import matplotlib.pyplot as plt
5 import matplotlib.image as img
6 import numpy as np
7 import cv2
8 import math
9
10 img = cv2.imread("../thes.png", 0)
11
12 w = img.shape[1] + 250
13 h = img.shape[0] + 250
14
15 tx = float(input("Translation in X: "))
16 ty = float(input("Translation in Y: "))
17
18 matrix = np.array([[1, 0, tx],
19                   [0, 1, ty]], dtype=np.float32)
20
21 translated_image = cv2.warpAffine(img, matrix, (w, h))
22
23 fig, axis = plt.subplots(1, 2, figsize=(7, 4))
24
25 axis[0].imshow(img)
26 axis[1].imshow(translated_image)
27 axis[1].set_title('Translated image')
28
29 plt.tight_layout()
30 plt.show()

```

Translation in X: 23

Translation in Y: 21

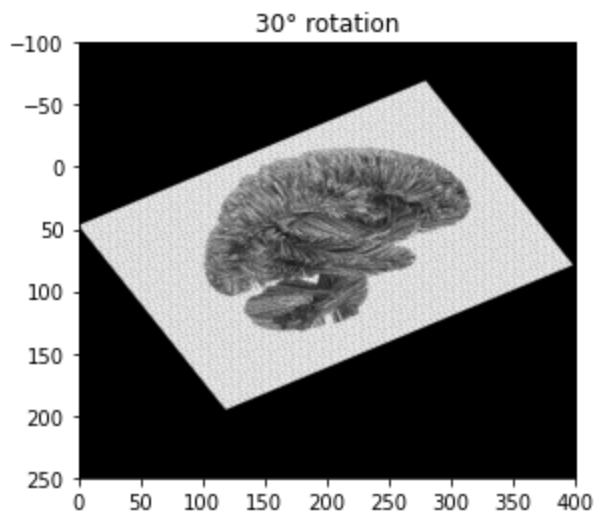


```

In [14]: 1 #2D 30° rotation
2 import matplotlib.pyplot as plt
3 import matplotlib.image as img
4 import numpy as np
5 import cv2
6 import math
7
8 img = cv2.imread("../thes.png",0)
9 h, w = img.shape
10
11 theta = int(input('Enter theta: '))
12 nimg = np.zeros((w+280, h+280), dtype = "uint8")
13 radians = math.radians(theta)
14
15 for i in range(h):
16     for j in range(w):
17         pix = img[i][j]
18         pnt = np.array([i,j]).reshape(-1,1)
19         rotm = np.array([math.cos(radians), -math.sin(radians), math.sin(r
20         rotm = rotm.reshape(2,2)
21         newmat = rotm @ pnt
22         xf = newmat[0][0]
23         yf = newmat[1][0]
24         x2 = int(xf+350)
25         y2 = int(yf)
26         nimg[x2,y2] = pix
27
28 print(nimg.shape)
29 fig, ax = plt.subplots()
30 ax.imshow(nimg, extent = [0, 400, 250, -100], cmap = 'gray')
31 plt.title("30° rotation")
32 plt.show()
33

```

Enter theta: 30
(832, 686)



```

In [15]: 1 #2D #scaling
2 import matplotlib.image as img
3 import numpy as npy
4 import matplotlib.pyplot as plt
5
6 m = img.imread("../thes.png");
7
8 w, h = m.shape[:2];
9
10 sx = float(input("sx:"))
11 sy = float(input("sy:"))
12
13 xNew = int(w * 2);
14 yNew = int(h * 2);
15
16 xScale = xNew/(w-1);
17 yScale = yNew/(h-1);
18
19 newImage = npy.zeros([xNew, yNew, 4]);
20
21 for i in range(xNew-1):
22     for j in range(yNew-1):
23         newImage[i + 1, j + 1] = m[1 + int(i / xScale), 1 + int(j / yScale)]
24
25 # Save the image after scaling
26 img.imsave('scaled1.png', newImage);
27 print(newImage.shape)
28 plt.imshow(newImage)

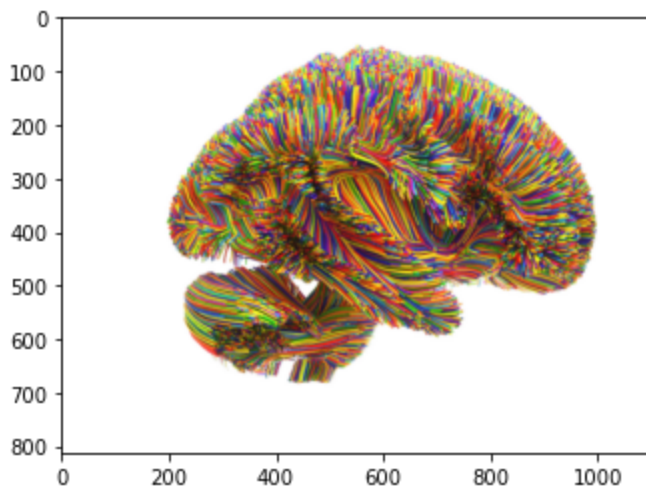
```

```

sx:4
sy:2
(812, 1104, 4)

```

Out[15]: <matplotlib.image.AxesImage at 0x17befe4af10>



In [5]:

```
1  #Colen Sutherland Line Clipping Algorithm
2  x1 = int(input('x1:'))
3  y1 = int(input('y1:'))
4  x2 = int(input('x2:'))
5  y2 = int(input('y2:'))
6
7  x_max = int(input("x_max: "))
8  y_max = int(input("y_max: "))
9  x_min = int(input("x_min: "))
10 y_min = int(input("y_min: "))
11
12 m = ((y2-y1) / (x2-x1))
13 m
14
15 intersection_points = []
16 if y_min <= y1 <= y_max:
17     x = x1 + ((y_min - y1) / m )
18     if x_min <= x <= x_max:
19         intersection_points.append((x, y_min))
20 if y_min <= y2 <= y_max:
21     x = x1 + ((y_max - y1) / m )
22     if x_min <= x <= x_max:
23         intersection_points.append((x, y_max))
24 if x_min <= x1 <= x_max:
25     y = y1 + (m * (x_min - x1))
26     if y_min <= y <= y_max:
27         intersection_points.append((x_min, y))
28 if x_min <= x2 <= x_max:
29     y = y1 + (m * (x_max - x1))
30     if y_min <= y <= y_max:
31         intersection_points.append((x_max, y))
32 print("Intersection points:", intersection_points)
33
34 if x > x_max and y > y_max:
35     top = 1
36     bottom = 0
37     right = 1
38     left = 0
39 elif x > x_max and y < y_min:
40     top = 0
41     bottom = 1
42     right = 1
43     left = 0
44 elif x > x_max and y_min < y < y_max:
45     top = 0
46     bottom = 0
47     right = 1
48     left = 0
49 elif x_min < x < x_max and y > y_max:
50     top = 1
51     bottom = 0
52     right = 0
53     left = 0
54 elif x_min < x < x_max and y < y_min:
55     top = 0
56     bottom = 1
57     right = 0
```

```
58     left = 0
59 elif x_min < x < x_max and y_min < y < y_max:
60     top = 0
61     bottom = 0
62     right = 0
63     left = 0
64 elif x < x_min and y > y_max:
65     top = 1
66     bottom = 0
67     right = 0
68     left = 1
69 elif x < x_min and y < y_min:
70     top = 0
71     bottom = 1
72     right = 0
73     left = 1
74 elif x < x_min and y_min < y < y_max:
75     top = 0
76     bottom = 0
77     right = 1
78     left = 0
79
80 print(top, bottom, right, left)
```

```
x1:-1
y1:7
x2:-4
y2:2
x_max: 2
y_max: 6
x_min: -3
y_min: 1
Intersection points: [(-1.6, 6), (-3, 3.6666666666666665)]
0 0 0 0
```

```

In [1]: 1 #weiler_atherton
2 window = [(0,3),(4,3),(4,0),(0,0)]
3
4 x_max = 4
5 y_max = 3
6 x_min = 0
7 y_min = 0
8
9 polygon = [(1,5),(4,5),(3,1),(2,1),(2,4),(1,1), (1,5)]
10
11 subject_list = []
12
13 for item in range(len(polygon) - 1):
14     x = polygon[item]
15     y = polygon[item+1]
16
17     in_x = x_min <= x[0] <= x_max and y_min <= x[1] <= y_max
18     in_y = x_min <= y[0] <= x_max and y_min <= y[1] <= y_max
19
20     if (not in_x and not in_y) or (in_x and in_y):
21         subject_list.append(x)
22     else:
23         subject_list.append(x)
24         subject_list.append('i')
25 print("subject list", subject_list)
26
27 clip_list = []
28
29 for item in range(len(window) - 1):
30     x = window[item]
31     y = window[item+1]
32
33     if (x and y) and (x not in clip_list and y not in clip_list):
34         clip_list.append(x)
35         for i in range(window[0][0], window[1][0]) or range(window[0][1],
36             clip_list.append(f"i{i}")
37         clip_list.append(y)
38
39 print("clip list: ", clip_list)
40

```

```

subject list [(1, 5), (4, 5), 'i', (3, 1), (2, 1), 'i', (2, 4), 'i', (1, 1),
'i']
clip list: [(0, 3), 'i0', 'i1', 'i2', 'i3', (4, 3), (4, 0), (0, 0)]

```

```

In [ ]: 1

```